Assignment 2

Introduction to NLP – Spring '26

Deadline: 5 March 2026, 11:59pm

General Instructions

1. The assignment must be implemented in **Python**.
2. Your grade will depend on both the validity of your output and your interpretation of the output and method used.
3. A single `.zip` file must be uploaded to the Moodle submission portal.
4. No extensions will be granted beyond the announced deadline.

1 Training Word Vectors

Many NLP systems employ modern distributional semantic algorithms, known as word embedding algorithms, to generate meaningful numerical representations for words. These algorithms aim to create embeddings where words with similar meanings are represented closely in a mathematical space. Word embeddings fall into two main categories: frequency-based and prediction-based. Frequency-based embeddings utilize various vectorization methods such as Count Vector, TF-IDF Vector, and Co-occurrence Matrix with a fixed context window. Prediction-based embeddings, exemplified by Word2vec, utilize models like Continuous Bag of Words (CBOW) and Skip-Gram (SG). However, the computational complexity of Word2vec's training algorithm can be high due to gradient calculations over the entire vocabulary. To tackle this challenge, variants such as Hierarchical Softmax output, Negative Sampling, and Subsampling of frequent words have been proposed. While "word vectors" and "word embeddings" are used interchangeably, "embedding" specifically denotes encoding a word's meaning into a lower-dimensional space

For each of the following algorithms, train word embeddings using the **Brown Corpus** (available through `nltk.corpus` provided by the NLTK library), which can be downloaded directly through the NLTK library. The learned embeddings must be saved as `.pt` files. Clearly report all hyperparameter choices.

1.1 SVD

Implement a word embedding model by first constructing a co-occurrence matrix followed by the application of Singular Value Decomposition (SVD). In your report, clearly justify the selection of all hyperparameters used in the process (e.g., context window size, weighting scheme, and embedding dimensionality).

1.2 Neural Word Embedding

Implement the Word2Vec model and train word vectors using **either** the Continuous Bag of Words (CBOW) model **or** the Skip-Gram model.

If CBOW is chosen, the model should predict a target word given its surrounding context words. If Skip-Gram is chosen, the model should predict surrounding context words given a current word. In both cases, **negative sampling must be implemented.**

2 Analysis: Are the Embeddings Fishy?

You will analyze the geometric properties of the embeddings you trained in Section 1 alongside one open-source, pre-trained word embedding (such as GloVe or fastText).

2.1 Task 1: Analogy Test (Semantic Capability)

A good embedding space should encode semantic relationships as linear offsets. For **all three** embedding variants (SVD, Word2Vec, Pre-trained), implement a script to answer analogy questions of the form $A : B :: C :?$ using the formula:

$$\text{maximize } \cos(x, \text{vec}(B) - \text{vec}(A) + \text{vec}(C))$$

Report the top 5 predicted words for the following analogies in your `report.pdf`:

1. *Paris : France :: Delhi : ?* (Syntactic/Capital)
2. *King : Man :: Queen : ?* (Semantic/Gender)
3. *Swim : Swimming :: Run : ?* (Syntactic/Tense)

2.2 Task 2: The Bias Check (Ethical Capability)

Word embeddings often capture harmful stereotypes present in the corpus. You must "audit" your models by measuring the cosine similarity of neutral profession words against gendered terms.

For the Pre-trained Embeddings only, compute and report the cosine similarity scores for the following pairs:

- **Pair A:** $\cos(\text{vec('doctor')}, \text{vec('man')})$ vs. $\cos(\text{vec('doctor')}, \text{vec('woman')})$
- **Pair B:** $\cos(\text{vec('nurse')}, \text{vec('man')})$ vs. $\cos(\text{vec('nurse')}, \text{vec('woman')})$
- **Pair C:** $\cos(\text{vec('homemaker')}, \text{vec('man')})$ vs. $\cos(\text{vec('homemaker')}, \text{vec('woman')})$

Note: This phenomenon and the specific methodology for quantifying it were first detailed by Bolukbasi et al. (2016)[1]. (While reading the paper is optional, it provides valuable context for the methodology used here.)

[1] Bolukbasi et al., "Man is to Computer Programmer as Woman is to Homemaker? Debiasing Word Embeddings": `https://arxiv.org/abs/1607.06520`

3 POS Tagging with Multi-Layer Perceptron

In this phase, you will utilize both SVD and Word2Vec word embeddings trained in 1, alongside one open-source, pre-trained word embedding (such as GloVe or fastText), to implement a Part-of-Speech (POS) tagger.
The task is to assign a grammatical category (tag) to every word in a text sequence. You will implement a Multi-Layer Perceptron (MLP) classifier that predicts the tag of a given word based on its embedding and the embeddings of its neighbors.

3.1 Dataset

You will use the **Brown Corpus** (available through `nltk.corpus` provided by the NLTK library), the same corpus used in Section 1.

- **Loading Data:** Use the `nltk.corpus.brown.tagged_sents(tagset='universal')` method.
- **Important:** You **must** specify `tagset='universal'` to retrieve the simplified set of 12 tags (e.g., NOUN, VERB, ADJ). Do not use the default complex tagset.
- **Splits:** Since NLTK does not provide pre-defined splits, you must randomly shuffle the sentences and split them manually:
  - **Training:** 80%
  - **Validation:** 10%
  - **Test:** 10%

3.2 Model Architecture

Since a standard MLP requires a fixed-size input vector, you cannot feed an entire variable-length sentence into the network at once. Instead, you must implement a **sliding window** approach (also known as a fixed-context window):

1. **Input Construction:** For a specific target word w_t that needs to be tagged, consider a context window of size C around it.
   - For example, if $C = 1$, the context includes the previous word, the current word, and the next word: $[w_{t-1}, w_t, w_{t+1}]$.
   - Retrieve the embedding vectors for these words and **concatenate** them to form a single input vector.
   - Handle boundary conditions (start/end of sentences) by using a special `<PAD>` token embedding.
2. **Network:** Pass this concatenated vector through **at least two** hidden layers with a non-linear activation function (e.g., ReLU or Tanh).
3. **Output:** The final layer should have a dimension equal to the number of unique POS tags in the dataset. Apply a Softmax function to obtain probabilities for each tag.

3.3 Experiments

You must train and evaluate your MLP tagger using the **three** different input embedding variants. You may choose to keep the embeddings frozen or fine-tune them during training (please justify your choice in the report).

1. **SVD Embeddings:** Initialize the embedding layer using the weights learned in Section 1.1.
2. **Word2Vec Embeddings:** Initialize using the weights (CBOW or Skip-Gram) learned in Section 1.2.
3. **Pre-trained Embeddings:** Initialize using pre-trained embeddings such as **GloVe** or **fastText**. You may use libraries like `gensim` or `torchtext` to load these.

3.4 Evaluation

Report the **Accuracy** and **Macro-F1 Score** on the test set for all three variants. Include a confusion matrix for your best-performing model in the report.

4 Analysis and Report

In your `report.pdf`, record the following:

1. **Embedding Comparison:** Compare the performance of SVD, Word2Vec, and Pre-trained embeddings. Did the pre-trained embeddings significantly outperform the ones you trained from scratch?
2. **Hyperparameters:** List the hyperparameters used for the MLP (e.g., window size, hidden layer size, learning rate).
3. **Error Analysis:** Provide 3-5 examples of sentences where the model predicted incorrect tags. Analyze why the model might have failed in these instances (e.g., ambiguity, unseen words).

5 Submission Format

Submit a compressed file named:

```
<roll_number>_A2.zip
```

Ensure your submission follows this structure strictly:

```
<roll_number>_A2.zip
|
|-- embeddings/
|   |-- svd.pt
|   |-- cbow.pt (or skipgram.pt)
|
|-- svd_embeddings.py    # Code for Section 1.1
|-- word2vec.py          # Code for Section 1.2 (CBOW or Skipgram)
|-- pos_tagger.py        # Code for Section 3 (MLP Training & Inference)
|
|-- README.md            # Instructions to run all scripts
|-- report.pdf           # Analysis and results
```

All explanations, assumptions, and analysis must be included in `report.pdf`. Instructions to run the code must be provided in `README.md`. Do not submit Jupyter notebooks, they will not be evaluated.

Note: Do not include the dataset or the heavy pre-trained embedding files in the zip. Push the models to hugging face or share a drive link.

Suggested Reading & Resources

- **The Illustrated Word2Vec** by Jay Alammar.
  `https://jalammar.github.io/illustrated-word2vec/`
- **Speech and Language Processing (3rd Ed.)** by Jurafsky & Martin.
  Chapter 5: Embeddings. `https://web.stanford.edu/~jurafsky/slp3/5.pdf`
- **Word2Vec:** Mikolov et al. (2013). *Efficient Estimation of Word Representations in Vector Space.*
  `https://arxiv.org/abs/1301.3781`
- **GloVe:** Pennington et al. (2014). *GloVe: Global Vectors for Word Representation.* Understanding the connection between count-based matrix factorization and window-based learning.
  `https://nlp.stanford.edu/pubs/glove.pdf`